Install proTRAC_2.4.3.pl from sourceforge: https://sourceforge.net/projects/protrac/ and PILFER collapsing script, make sure the command fits latest Python version

USE same procedures (first 3 steps from PILFER for collapse)

conda activate pilfer_env #create and activate virtual environment with pilfer
cd your_sample_directory_path

Step 1: Make directory for each tool's output files:

```
mkdir -p collapse
mkdir -p mm10_sam
mkdir -p logs/bowtie1
mkdir -p logs/pirna_map/
mkdir -p logs/cutadapt/
mkdir -p pirna_sam
mkdir -p pirna_sam/filter
mkdir -p bed/putative
mkdir -p bed/ncrna_subtract_putative
mkdir -p bed/cluster
mkdir -p bed/TE_origin
mkdir -p TE_profile
```

Check if all created successfully:

```
ls -d collapse mm10_sam logs/bowtie1 logs/pirna_map logs/cutadapt pirna_sam
pirna_sam/filter bed/putative bed/ncrna_subtract_putative bed/cluster bed/TE_origin TE_profile
```

Step 2: Define general inputs/output and paths

```
list=("your_treatment_sample1.fastqsanger"
"your_treatment_sample2.fastqsanger"
"your_treatment_sample3.fastqsanger"
"your_treatment_sample4.fastqsanger"
"your_treatment_sample5.fastqsanger"
"your_control_sample1.fastqsanger" "your_control_sample2.fastqsanger"
"your_control_sample3.fastqsanger" "your_control_sample4.fastqsanger"
"your_control_sample4.fastqsanger")
sample_dir_prefix="./"              # Current directory for raw sample
files
out_dir_prefix="./"                 # Current directory for output
```

```
tools_dir="your path to tools directory inside of PILFER"# Path to
tools directory inside PILFER
num_sample=10                          # Total number of samples
```

Step 3: Run collapse.py for Each Sample

```
for prefix in "${list[@]}"
do
   echo "Running Collapse for $prefix"
   python "$tools_dir"collapse.py -i "$sample_dir_prefix$prefix.gz" -o
"$out_dir_prefix"collapse/"$prefix.fa" --format fastq
done
```

Note the output file in : 'your_sample_directory_path/collapse/…"

Then to protrac steps

```
conda activate bioinfo #activate virtual environment
cd your_sample_directory_path
```

```
#Build index (time-consuming part and split the whole genome into buckets)
#conda install -c bioconda bowtie
bowtie-build GRCm39.fa GRCm39p

#checking output
ls -lh GRCm39p*
```

```
# Define directories
genome_index="your_index_path/GRCm39p"   # Index for Bowtie
input_dir="your_collapsed_file_path/collapse"            # Directory with compressed input files
output_dir="${input_dir}/mapping_results"          # Output directory for SAM files

mkdir -p "${output_dir}"

#Mapping
```

```
for file in your_treatment_sample_1.fastqsanger.fa.gz
your_treatment_sample_2.fastqsanger.fa.gz your_treatment_sample_3.fastqsanger.fa.gz
your_treatment_sample_4.fastqsanger.fa.gz your_treatment_sample_5.fastqsanger.fa.gz \
        your_control_sample_1.fastqsanger.fa.gz your_control_sample_2.fastqsanger.fa.gz
your_control_sample_3.fastqsanger.fa.gz your_control_sample_4.fastqsanger.fa.gz
your_control_sample_5.fastqsanger.fa.gz; do
   echo "Mapping ${file} to genome using Bowtie..."
   bowtie -f -v 2 -x "${genome_index}" "${input_dir}/${file}" -S
"${output_dir}/${file%.fastqsanger.fa.gz}.sam"
done


ls -lh "${output_dir}"/*.sam
```

Note output files in
'your_sample_path/collapse/mapping_results/your_treatment_sample_1.sam'

***# Convert SAM to BAM, then sort it for later use to intersect with merged gtf file**

```
cd your_sample_path/collapse/mapping_results

# Set the directory containing the SAM files
input_dir="your_sample_path/collapse/mapping_results"
# Set the directory where you want to save the BAM files
output_dir="your_sample_directory_path"
# Navigate to the directory containing the SAM files
cd "$input_dir"

for samfile in *.sam; do
   bamfile="${samfile%.sam}.bam"
   echo "Converting $samfile to BAM format..."
   # Convert and sort SAM file, outputting to the specified output directory
   samtools view -Sb $samfile | samtools sort -o "${output_dir}/${bamfile}"
   echo "Indexing ${output_dir}/${bamfile}..."
   # Index the sorted BAM file
   samtools index "${output_dir}/${bamfile}"
done
```

#Filter piRNA information from BAM for 21-33bp length, these subjunc_results bams will come from the proTRAC_DE.R script, and then will need to be processed in bash before returning to R.

```
bam_files=("subjuncts_result1.bam" "subjuncts_result2.bam" "subjuncts_result3.bam"
```

```
        "subjuncts_result4.bam" "subjuncts_result5.bam" "subjuncts_result6.bam"
        "subjuncts_result7.bam" "subjuncts_result8.bam" "subjuncts_result9.bam"
        "subjuncts_result10.bam")

for bam_file in "${bam_files[@]}"
do
   echo "Processing $bam_file"
   # Add your processing commands here, e.g., filtering reads by length
   samtools view -h "$bam_file" | awk 'length($10) >= 21 && length($10) <= 33 || $1 ~ /^@/' |
samtools view -b > "filtered_piRNA${bam_file}"
done
```

proTRAC steps

```
cd "your_sample_directory_path"

# Path to the directory containing outputs and genome file
output_dir="your_sample_path/collapse/mapping_results"
genome_fasta="your_genome_path/GRCm39.fa"

for samfile in "${output_dir}"/*.sam; do
   # Extract the base name of the file without the .sam extension
   base_name=$(basename "${samfile}" .sam)

   # Run proTRAC with parameters for an unbiased comparison with PILFER
   perl your_protrac_path/proTRAC_2.4.3.pl \
      -map "${samfile}" \
      -genome "${genome_fasta}" \
      -pimin 21 \
      -pimax 33 \
      -1Tor10A 0.75 \
      -clstrand 0.75 \
      -distr 1-100 \
      -out "${output_dir}/${base_name}_output.txt"
done
```

Check output

```
ls -lh your_sample_directory_path/collapse/mapping_results/
```

```
find your_sample_directory_path/collapse/mapping_results/ -name "*.html"

find your_sample_directory_path -type f \( -name "*.html" -or -name "*.gtf" -or -name "*.fasta" \)
-exec ls -lh {} +

find your_sample_directory_path -name "*.html" -exec ls -lh {} +

find your_sample_directory_path -name "*.fasta" -exec ls -lh {} +

find your_sample_directory_path -name "*.gtf" -exec ls -lh {} +
```

#Merge clusters.gtf files:

```
cat \
"your_treatment_sample1_prefix_path/clusters.gtf" \
"your_treatment_sample2_prefix_path/clusters.gtf" \
"your_treatment_sample3_prefix_path/clusters.gtf/clusters.gtf" \
"your_treatment_sample4_prefix_path/clusters.gtf/clusters.gtf" \
"your_treatment_sample5_prefix_path/clusters.gtf/clusters.gtf" \
"your_control_sample6_prefix_path/clusters.gtf/clusters.gtf" \
"your_control_sample7_prefix_path/clusters.gtf/clusters.gtf" \
"your_control_sample8_prefix_path/clusters.gtf/clusters.gtf" \
"your_control_sample9_prefix_path/clusters.gtf/clusters.gtf" \
"your_control_sample10_prefix_path/clusters.gtf/clusters.gtf" \
> combined.gtf


#Add gene id to gtf
awk 'BEGIN{FS=OFS="\t"} {
    gsub(/.*piRNA cluster no: ([^;]+);.*/, "gene_id \"cluster_"$9"\";", $9);
    print;
}' your_file_path/combined.gtf > your_file_path/modified.gtf
```

#these two have chromosome info, start, end, strand length full info****

Count merged gtf across each sample

#Convert merged gtf to bed
```
awk 'BEGIN {FS=OFS="\t"} $3 == "piRNA_cluster" {print $1, $4-1, $5, $9, ".", $7}'
your_file_path/modified.gtf > your_file_path/modified.bed
```

#these two have chromosome info, start, end, strand length full info****
##Note each line here is a single cluster, the file here is combined from all samples from top to bottom, merging overlapped region. While the “geneID no. are not modified” so the clusters with same “gene id no.” here are diff clusters. The clusters are assigned based on the order from top to bottom as cluster 1 to 1800

```
for bam in your_treatment_sample1_path.bam \
          your_treatment_sample2_path.bam \
          your_treatment_sample3_path.bam \
          your_treatment_sample4_path.bam \
         your_treatment_sample5_path.bam \
          your_control_sample1_path.bam \
         your_control_sample2_path.bam \
          your_control_sample3_path.bam \
           your_control_sample4_path.bam \
         your_control_sample5_path.bam; do
   sample=$(basename "$bam" .bam)
   bedtools intersect -a your_sample_directory_path/modified.bed -b "$bam" -c >
"your_sample_directory_path/${sample}_counts.txt"
done
```

#combine to a count matrix
Create an output file for the header, this provide the gene id info before rename to cluster
```
output_file="your_sample_directory_path/piRNA_cluster_count_matrix.txt"
```

Write the header to the output file with sample name prefixes
```
echo -e
"Cluster_ID\ttreatment_sample1\ttreatment_sample2\ttreatment_sample3\ttreatment_sample4\tt
reatment_sample5\tcontrol_sample1\tcontrol_sample2\tcontrol_sample3\tcontrol_sample4\tcont
rol_sample5" > "$output_file"
```

Extract the cluster IDs from the first file and prepare for combining counts
```
cut -f4 "sample_directory_path_counts.txt" | awk '{print $1}' > temp_cluster_ids.txt
```

Combine the counts from all the sample files column-wise

```
paste temp_cluster_ids.txt \
    <(cut -f7 "treatment_sample1_path_counts.txt") \
    <(cut -f7 "treatment_sample2_path_counts.txt") \
    <(cut -f7 "treatment_sample3_path_counts.txt") \
    <(cut -f7 "treatment_sample4_path_counts.txt") \
    <(cut -f7 "treatment_sample5_path_counts.txt") \
    <(cut -f7 "control_sample1_path_counts.txt") \
    <(cut -f7 "control_sample2_path_counts.txt") \
    <(cut -f7 "control_sample3_path_counts.txt") \
    <(cut -f7 "control_sample4_path_counts.txt") \
    <(cut -f7 "control_sample5_path_counts.txt") \
    >> "$output_file"
```

Rename gene id and Order by Cluster ID

```
# Define input and output files
input_file="piRNA_cluster_count_matrix.txt"
output_file="piRNA_cluster_count_matrix_renamed.txt"

# Process the file to rename gene_id values
awk 'BEGIN {FS=OFS="\t"}
NR == 1 {print; next}
{ $1 = "cluster_" NR-1; print }' "$input_file" > "$output_file"
```

USE this for DE

Then find start end chromosome info of DEs from modified BED file based on line #, not the geneID no. Then use it for intersection

Save and move to proTRAC_DE.R

Intersection for target gene prediction

Save the DE csv file with chr, start , end, into txt format and intersect wc gtf file

```
#sudo apt install -y bedtools
#conda install -c bioconda bedtools


# Convert Gene annotation GTF to BED and extract gene names
awk -F'\t' '$3 == "gene" {
```

```
    gsub(/.*gene_name "/, "", $9);  # Remove everything before gene_name "
    gsub(/";.*/, "", $9);  # Remove everything after ";
    print $1 "\t" $4-1 "\t" $5 "\t" $9  # Print results
}' gencode.vM37.chr_patch_hapl_scaff.annotation.gtf > genes.bed
```

#Convert DE txt file (with chr, start, end) to bed for intersection
Manually add column title to the DE csv file (saved from R) with format of

Gene	Chromosome	Start	End	baseMean	log2FoldChange	lfcSE	stat	pvalue	adj.P.Val
cluster_75	chr14	65971729	65981528	115.8080571	-2.52433854	0.464866271	-5.430246715	5.63E-08	4.78E-05
cluster_272	chr14	65971767	65977032	68.57700016	-2.368685683	0.444926496	-5.323768538	1.02E-07	4.78E-05
cluster_678	chr14	65973149	65982034	108.2960147	-2.518720078	0.474937201	-5.303269725	1.14E-07	4.78E-05
cluster_1675	chr14	65971194	65981513	116.6338346	-2.481684086	0.468851127	-5.293117455	1.20E-07	4.78E-05

Find chromosome, start, end info , cluster ID match with row # from modified.bed

```
awk 'NR > 1 {print $2 "\t" $3 "\t" $4}' "differentially_expressed_clusters.txt" >
clusters.bed
```

#With M10 GTF (based on GRCm39)

```
bedtools intersect -a clusters.bed -b genes.bed -wa -wb > intersected_genes.bed
```

For top50 DE coordinate and gene info:

Additional: generate top 50 expressed clusters wc their coordinate and gene info (March09th2025)use saved csv from DE res result, keep top 50 (based on FDR)
Following the
(# Convert Gene annotation GTF to BED and extract gene names (done above)

```
awk -F'\t' '$3 == "gene" {
    gsub(/.*gene_name "/, "", $9);  # Remove everything before gene_name "
    gsub(/";.*/, "", $9);  # Remove everything after ";
    print $1 "\t" $4-1 "\t" $5 "\t" $9  # Print results
}' gencode.vM10.annotation.gtf > genes.bed
```

)
Next make the Top 50 expressed piRNA clusters CSV file into “Top 50 expressed piRNA clusters with coordinate structure” and exported to txt file

Clusters	Chromosome	Start	End	FDR	average number of piRNA hits	log2FoldChange

cluster_394	0.055829967	-0.379153988
cluster_696	0.055829967	-0.39439832
cluster_466	0.059610019	-0.371477387

Then extract information of Chromosome, Start, End from the modified.bed, here each cluster ID number match with the row number in modified bed, for example cluster_394 is the row 394 in modified.bed, so extract the chromosome, start, end from row 394 in modified.bed and fill into the table above

*The **line (row) number** in modified.bed is the unique "cluster_N" you use in the stats table* (row 394 → cluster_394, row 696 → cluster_696, etc.).

So instead of parsing the annotation, simply attach the current **NR (awk record number)** to each BED line:

```
awk -F'\t' '
   BEGIN {OFS="\t"}
   /^[#;]/ {next}               # skip header or comment lines if any
   {
      print "cluster_" NR, $1, $2, $3   # cluster_N  chrom  start  end
   }
' modified.bed > cluster_coords.tsv

# 1) sort the stats body
tail -n +2 "Top 50 expressed piRNA clusters with coordinate structure.txt" \
| sort -k1,1 > stats_body.tsv

# 2) sort the new coordinate table
sort -k1,1 cluster_coords.tsv > coords.sorted

# 3) join on column 1 (cluster ID)
join -t $'\t' -1 1 -2 1 stats_body.tsv coords.sorted |
awk -F'\t' 'BEGIN{OFS="\t"}
  {
   # joined layout: Clusters emptyChrom emptyStart emptyEnd FDR avgHits log2FC
chrom start end
   chrom=$(NF-2); start=$(NF-1); end=$NF
   fdr=$(NF-5); avg=$(NF-4); lfc=$(NF-3)
   print $1, chrom, start, end, fdr, avg, lfc
  }' \
```

```
> body_filled.tsv

# 4) add header
echo -e "Clusters\tChromosome\tStart\tEnd\tFDR\taverage number of piRNA
hits\tlog2FoldChange" \
  > Top50_with_coords.txt
cat body_filled.tsv >> Top50_with_coords.txt
```

Then intersect with count matrix to get average piRNA hits for each cluster

Compute the mean hits for every cluster

```
# Make a two-column table:  cluster_ID   mean_hits
awk -F'\t' '
   NR==1 {next}                    # skip header
   {
      sum = 0;
      for (i = 2; i <= NF; i++) sum += $i;
      printf "%s\t%.4f\n", $1, sum / (NF-1);
   }
'  piRNA_cluster_count_matrix_renamed.txt \
| sort -k1,1  >  mean_hits.sorted
```

Merge the mean hits into Top50_with_coords.txt

```
# Strip header, sort body on Clusters
tail -n +2  Top50_with_coords.txt | sort -k1,1  > coords_body.sorted

# Join on cluster ID, then place the new meanHits field into the right slot
join -t $'\t' -1 1 -2 1  coords_body.sorted  mean_hits.sorted \
| awk -F'\t' 'BEGIN{OFS="\t"}
     {
       # joined layout:
       # $1  Clusters
       # $2  Chromosome   $3 Start   $4 End
       # $5  FDR          $6 oldAvg(blank)  $7 log2FC
       # $8  meanHits (from join)
       print $1, $2, $3, $4, $5, $8, $7   # new order with meanHits in column 6
     }' \
> body_final.tsv
```

Add header and save

```
echo -e "Clusters\tChromosome\tStart\tEnd\tFDR\taverage number of piRNA
hits\tlog2FoldChange" \
  > Top50_with_coords_and_hits.txt

cat body_final.tsv >> Top50_with_coords_and_hits.txt
```

(Optional CSV version) so can fliter the FDR smallest to largest for top 50

```
tr '\t' ',' < Top50_with_coords_and_hits.txt \
  > Top50_with_coords_and_hits.csv
```

SORT FDR IN CSV AND USE THIS to intersect gtf file (genes.bed) FOR MAKING THE TABLE

Create topclusters.bed

```
awk -F'\t' 'NR>1 {              # skip header
    printf "%s\t%s\t%s\t%s\n",  $2, $3, $4, $1    # chr  start  end  clusterID
}'  Top50_with_coords_and_hits.txt \
> topclusters.bed
```

Sort both BEDs (required for bedtools on large files)

```
sort -k1,1 -k2,2n  topclusters.bed  > topclusters.sorted.bed
sort -k1,1 -k2,2n  genes.bed        > genes.sorted.bed
```

Intersect*

```
bedtools intersect \
   -a topclusters.sorted.bed \
   -b genes.sorted.bed \
   -wa -wb \
   > topintersected_genes.bed
```

Generate the CSV file into structure of Clusters,Chromosome,Start,End,FDR,average number of piRNA hits,log2FoldChange,Genes
If a given cluster overlaps multiple genes, they are collapsed into one
cell, separated by commas (e.g. GeneA,GeneB,GeneC).

```
awk -F'\t' '
   {
      cluster = $4;                  # 4th field in intersect = cluster_ID
      gene    = $NF;                   # last field  = gene symbol/ID
      # add gene if not already present (avoid duplicates)
      if (index(","genes[cluster]",", ","gene",") == 0) {
         if (genes[cluster] == "") genes[cluster] = gene;
         else                      genes[cluster] = genes[cluster]","gene;
```

```
        }
    }
    END { for (c in genes) print c "\t" genes[c] }
'  topintersected_genes.bed \
| sort -k1,1  > cluster_genes.tsv
```

Sort the main Top-50 table (body only)

```
tail -n +2 Top50_with_coords_and_hits.txt | sort -k1,1 > stats_body.sorted
```

```
#Clean CR characters out of both source files
tr -d '\r' < stats_body.sorted   > stats_body.clean
tr -d '\r' < cluster_genes.tsv   > cluster_genes.clean
#Join stats + gene list
awk -F'\t' '
    # ---------- first pass: cluster_genes.clean -------------------------
    FNR==NR { genes[$1] = $2; next }

    # ---------- second pass: stats_body.clean ---------------------------
    NR==1 {
        print "Clusters\tChromosome\tStart\tEnd\tFDR\taverage number of piRNA
hits\tlog2FoldChange\tGenes"
        next
    }
    {
        g = ($1 in genes ? genes[$1] : "");
        print $0 "\t" g
    }
' cluster_genes.clean stats_body.clean \
> Top50_with_coords_and_genes.txt       # <-- TAB-delimited

# Convert TAB → CSV  (fix header, quote Genes column)
awk -F'\t' 'BEGIN{OFS=","}
    NR==1 { gsub(/\t/, ",", $0); print; next }   # header → commas
    {
        genes = "\"" $NF "\"";   # quote Genes field
        NF--;                    # drop Genes temporarily
        print $0, genes;         # other columns + quoted Genes
    }
' Top50_with_coords_and_genes.txt \
> Top50_with_coords_and_genes.csv
```

Last add header to the txt and csv files to create table
Add header to the TXT (TAB-delimited) file

```
printf "Clusters\tChromosome\tStart\tEnd\tFDR\taverage number of piRNA hits\tlog2FoldChange\tGenes\n" \
  | cat - Top50_with_coords_and_genes.txt \
  > Top50_with_coords_and_genes.tmp && mv Top50_with_coords_and_genes.tmp Top50_with_coords_and_genes.txt
```

Add header to the CSV (comma-delimited) file

```
printf "Clusters,Chromosome,Start,End,FDR,average number of piRNA hits,log2FoldChange,Genes\n" \
  | cat - Top50_with_coords_and_genes.csv \
  > Top50_with_coords_and_genes.tmp && mv Top50_with_coords_and_genes.tmp Top50_with_coords_and_genes.csv
```

Done

#Intersect

```
bedtools intersect -a topclusters.bed -b genes.bed -wa -wb > topintersected_genes.bed
```

Can use this bed file info to fill up previous excel directly

#Aggregate Gene Names for Each Cluster

```
awk '{
    cluster = $4;
    gene = $8;
    # If this cluster already has a gene, only add if not already present.
    if (cluster in genes) {
        if (index(genes[cluster], gene) == 0)
            genes[cluster] = genes[cluster] "," gene;
    } else {
        genes[cluster] = gene;
    }
}
END {
    for (c in genes)
        print c "\t" genes[c]
}' topintersected_genes.bed > cluster_to_genes.tsv
```

#Merge the Aggregated Genes Back into the Original CSV

```
awk -F',' 'BEGIN {
    OFS=",";
    # Load cluster-to-genes mapping from cluster_to_genes.tsv
    while ((getline < "cluster_to_genes.tsv") > 0) {
        mapping[$1] = $2;
    }
    close("cluster_to_genes.tsv");
}
NR==1 {
    # Print header with extra "Gene" column
    print $0, "Gene";
    next;
}
{
    cluster = $1;
    gene = (cluster in mapping) ? mapping[cluster] : "";
    print $0, gene;
}' "Top 50 expressed piRNA clusters.csv" >
"Top_50_expressed_piRNA_clusters_with_genes.csv"
```

New Method–no DE but Merge Treatment and control GTF separately and find those unique clusters for functional study

```
cd your_sample_directory_path
```

Merge clusters.gtf files for treatment samples:

```
cat \
"treatment_sample1_path/clusters.gtf" \
"treatment_sample2_path/clusters.gtf" \
"treatment_sample3_path/clusters.gtf" \
"treatment_sample4_path/clusters.gtf" \
"treatment_sample5_path/clusters.gtf" \
> yourTREATMENTcombined.gtf
```

#Add gene id to gtf

```
awk 'BEGIN{FS=OFS="\t"} {
    gsub(/.*piRNA cluster no: ([^;]+);.*/, "gene_id \"cluster_"$9"\";", $9);
    print;
}' /your_sample_directory_path/yourTREATMENTcombined.gtf >
your_sample_directory_path/yourTREATMENTmodified.gtf
```

#Convert merged gtf to bed

```
awk 'BEGIN {FS=OFS="\t"} $3 == "piRNA_cluster" {print $1, $4-1, $5, $9, ".", $7}'
your_sample_directory_path/yourTREATMENTmodified.gtf >
your_sample_directory_path/yourTREATMENTmodified.bed
```

Merge GTF files:

```
cat \
"control_sample1_path/clusters.gtf" \
"control_sample2_path/clusters.gtf" \
"control_sample3_path/clusters.gtf" \
```

```
"control_sample4_path/clusters.gtf" \
"control_sample5_path/clusters.gtf" \
> yourCONTROLcombined.gtf



#Add gene id to gtf
awk 'BEGIN{FS=OFS="\t"} {
    gsub(/.*piRNA cluster no: ([^;]+);.*/, "gene_id \"cluster_"$9"\";", $9);
    print;
}' your_sample_directory_path/yourCONTROLcombined.gtf >
your_sample_directory_path/yourCONTROLmodified.gtf

#Convert merged gtf to bed
awk 'BEGIN {FS=OFS="\t"} $3 == "piRNA_cluster" {print $1, $4-1, $5, $9, ".", $7}'
your_sample_directory_path/yourCONTROLmodified.gtf >
your_sample_directory_path/yourCONTROLmodified.bed

#Check overlap: chromosome first then start end
sort -k1,1 -k2,2n yourTREATMENTmodified.bed   > yourTREATMENTmodified.sorted.bed
sort -k1,1 -k2,2n yourCONTROLmodified.bed > yourCONTROLmodified.sorted.bed

#chromosome check
echo "LPS chroms:";   cut -f1 yourTREATMENTmodified.sorted.bed   | sort -u
echo "CTRL chroms:";   cut -f1 yourCONTROLmodified.sorted.bed | sort -u


echo "Chroms in TREATMENT but not in CTRL:"
comm -23 <(cut -f1 yourTREATMENTmodified.sorted.bed | sort -u) <(cut -f1
yourCONTROLmodified.sorted.bed  | sort -u)

echo "Chroms in CTRL but not in TREATMENT:"
comm -13 <(cut -f1 yourTREATMENTmodified.sorted.bed | sort -u) <(cut -f1
yourCONTROLmodified.sorted.bed | sort -u)

#Any overlap (same chromosome automatically)
bedtools intersect -a yourTREATMENTmodified.sorted.bed -b
yourCONTROLmodified.sorted.bed -wa -wb \
  > TREATMENT_vs_CTRL.overlap.tsv

#Classify overlap as WITHIN / INCLUDES / OVERLAP_ONLY

#Output columns:
# chr treatment_start treatment_end relationship treatment_attr ctrl_start ctrl_end ctrl_attr
```

```
awk 'BEGIN{OFS="\t"}
{
  # TREATMENT (A)
  aS=$2; aE=$3;
  # CTRL (B) start/end are columns 8/9 because -wa -wb appends B after A
  bS=$8; bE=$9;

  rel="OVERLAP_ONLY";
  if (aS>=bS && aE<=bE) rel="TREATMENT_WITHIN_CTRL";
  else if (aS<=bS && aE>=bE) rel="TREATMENT_INCLUDES_CTRL";

  print $1,aS,aE,rel,$4, bS,bE,$10
}' TREATMENT_vs_CTRL.overlap.tsv > TREATMENT_vs_CTRL.classified.tsv
```

1)LPS clusters that are NOT in control at all (no overlap). USE this for gene intersection and functional study 0 clusters

```
bedtools intersect -a yourTREATMENTmodified.sorted.bed -b
yourCONTROLmodified.sorted.bed -v \
  > TREATMENT_unique_noOverlap.bed
```

1)Control clusters that are NOT in LPS at all (no overlap) !! USE this for gene intersection and functional study 18 clusters

```
bedtools intersect -a yourCONTROLmodified.sorted.bed -b
yourTREATMENTmodified.sorted.bed -v \
  > Control_unique_noOverlap.bed
```

1) “TREATMENT fully contained in any control” 940 clusters

```
awk '$4=="TREATMENT_WITHIN_CTRL"' TREATMENT_vs_CTRL.classified.tsv >
TREATMENT_within_ctrl.tsv
```

3) “TREATMENT includes any control” 0 clusters

```
awk '$4=="TREATMENT_INCLUDES_CTRL"' TREATMENT_vs_CTRL.classified.tsv >
TREATMENT_includes_ctrl.tsv
```

USE THIS ONE TREATMENT_unique_noOverlap.bed **FOR GENE INTERSECTION AS upregulated cluster, use** Control_unique_noOverlap.bed **FOR GENE INTERSECTION as downregulated clusters**

```
#change to your directory

# Convert Gene annotation GTF to BED and extract gene names (done; only once)
awk -F'\t' '$3 == "gene" {
    gsub(/.*gene_name "/, "", $9);  # Remove everything before gene_name "
    gsub(/";.*/, "", $9);  # Remove everything after ";
    print $1 "\t" $4-1 "\t" $5 "\t" $9  # Print results
}' gencode.vM37.chr_patch_hapl_scaff.annotation.gtf  > genes.bed

#Rebuild to Bed6 format
awk -F'\t' 'BEGIN{OFS="\t"} $3=="gene" {
    g=$9
    sub(/.*gene_name "/, "", g)
    sub(/".*/, "", g)
    print $1, $4-1, $5, g, ".", $7
}' your_file_path/gencode.vM37.chr_patch_hapl_scaff.annotation.gtf \
| sort -k1,1 -k2,2n \
> your_file_path/genes.bed



#Intersect the bed file with this
cd your_bed_file_directory

#With gencode.vM37.chr_patch_hapl_scaff.annotation.gtf (based on GRCm39)
bedtools intersect \
  -a TREATMENT_unique_noOverlap.bed \
  -b your_file_path/genes.bed \
  -s \
  -wa -wb \
  > intersected_genes.bed

bedtools intersect \
  -a Control_unique_noOverlap.bed \
  -b your_file_path/genes.bed \
  -s \
  -wa -wb \
  > Controlintersected_genes.bed


 #Collapse genes from same clusters into same row:Create a stable cluster key for each
TREATMENT interval
awk 'BEGIN{FS=OFS="\t"} {key=$1":"$2"-"$3":"$6; print key,$0}'
TREATMENT_unique_noOverlap.bed \
```

```
> TREATMENT.keyed.tsv

awk 'BEGIN{FS=OFS="\t"} {key=$1":"$2"-"$3":"$6; print key,$0}' Control_unique_noOverlap.bed
\
> Control.keyed.tsv


#Collapse gene names per cluster (comma-separated)
awk 'BEGIN{FS=OFS="\t"}
{
  key=$1":"$2"-"$3":"$6
  gene=$10
  if (gene=="") gene="NA"

  # build comma-separated unique list per key
  if (!( (key SUBSEP gene) in seen )) {
    seen[key SUBSEP gene]=1
    genes[key] = (genes[key]=="" ? gene : genes[key]","gene)
  }
}
END{
  for (k in genes) print k, genes[k]
}' intersected_genes.bed > genes_collapsed.tsv

#Join back to LPS, output EXACTLY 8 rows + header
awk 'BEGIN{FS=OFS="\t"}
NR==FNR {g[$1]=$2; next}
BEGIN{
  print "Cluster_ID","Chromosome","Start","End","Genes","Score","Strand"
}
{
  key=$1":"$2"-"$3":"$6
  cid="cluster_" NR
  gene_list = (key in g ? g[key] : "NA")
  print cid, $1, $2, $3, gene_list, $5, $6
}' genes_collapsed.tsv TREATMENT_unique_noOverlap.bed >
TREATMENT_clusters_withGenes.tsv


#THIS CAN BE CONSIDERED  UPREGULATED CLUSTERS
```

```
#Control
awk 'BEGIN{FS=OFS="\t"}
{
  key=$1":"$2"-"$3":"$6
  gene=$10
  if (gene=="") gene="NA"

  # build comma-separated unique list per key
  if (!( (key SUBSEP gene) in seen )) {
    seen[key SUBSEP gene]=1
    genes[key] = (genes[key]=="" ? gene : genes[key]","gene)
  }
}
END{
  for (k in genes) print k, genes[k]
}' Controlintersected_genes.bed > Controlgenes_collapsed.tsv

#Join back to Control, output EXACTLY 8 rows + header

awk 'BEGIN{FS=OFS="\t"}
NR==FNR {g[$1]=$2; next}
BEGIN{
  print "Cluster_ID","Chromosome","Start","End","Genes","Score","Strand"
}
{
  key=$1":"$2"-"$3":"$6
  cid="cluster_" NR
  gene_list = (key in g ? g[key] : "NA")
  print cid, $1, $2, $3, gene_list, $5, $6
}' Controlgenes_collapsed.tsv Control_unique_noOverlap.bed > Control_clusters_withGenes.tsv


#THIS CAN BE CONSIDERED  DOWNREGULATED CLUSTERS


#USE them separately for functional study
```